k-Nearest Neighbours Classification

2026-04-17

Introduction

k-nearest-neighbours classification assigns each test observation the majority class among its k nearest training points in feature space. It is the simplest instance-based learning method: there is no training step beyond storing the data, and prediction is a search-and-vote operation. kNN is competitive on low-dimensional, well-separated data and serves as a useful baseline against which more complex methods are compared. Performance depends critically on the distance metric and on k, both of which require attention.

Prerequisites

A working understanding of distance metrics (Euclidean, Manhattan, Mahalanobis), train-test splits, and the bias-variance trade-off in supervised learning.

Theory

For a query point x^* and training set $\{(x_i, y_i)\}$, the prediction is the mode of the labels of the k nearest neighbours: $\hat{y}(x^*) = \text{mode}\{y_i : x_i \in N_k(x^*)\}$. Distance-weighted variants weight votes inversely by distance, giving closer neighbours more influence and often improving boundary smoothness.

The bias-variance trade-off in k is direct: small k produces high-variance, low-bias decisions (each prediction depends on a few neighbours and adapts to local noise); large k produces high-bias, low-variance decisions (each prediction averages over a larger neighbourhood and misses local structure). Cross-validation chooses the optimal k.

The choice of distance metric matters: Euclidean is the default, Manhattan is more robust to outliers, Mahalanobis accounts for feature covariance, and Gower distance handles mixed numeric-categorical inputs.

Assumptions

Features are appropriately scaled (distances are scale-sensitive); labels are sufficiently clean for local majority voting to be informative; the chosen distance metric reflects meaningful similarity.

R Implementation

```
library(class)

set.seed(2026)
n <- 400
x1 <- rnorm(n); x2 <- rnorm(n)
y <- factor(ifelse(x1^2 + x2^2 < 1, "in", "out"))
df <- data.frame(x1 = scale(x1)[, 1], x2 = scale(x2)[, 1], y = y)

idx <- sample(n, n * 0.7)
train <- df[idx, ]; test <- df[-idx, ]

# kNN at different k
```

```r
accs <- sapply(c(1, 5, 15, 51), function(k) {
  pred <- knn(train[, 1:2], test[, 1:2], train$y, k = k)
  mean(pred == test$y)
})
setNames(accs, paste0("k=", c(1, 5, 15, 51)))
```

Output & Results

Test accuracy across a grid of k values reveals the U-shaped bias-variance curve: very small k overfits training noise, very large k oversmooths the decision boundary, and a moderate k minimises test error.

Interpretation

A reporting sentence: "k-NN classification achieved best test accuracy of 0.91 at $k = 15$ on the circular-decision-boundary task; $k = 1$ over-fit to noise with accuracy 0.82, while $k = 51$ over-smoothed and dropped to 0.80. Cross-validation across k confirmed $k = 15$ as the optimum." Always state the chosen k, the distance metric, and the test-set accuracy.

Practical Tips

- Always standardise features before kNN; unscaled features with different units dominate the distance computation and bias predictions.
- Tune k by cross-validation; the U-shaped accuracy-vs-k curve has a clear optimum on most datasets.
- For high-dimensional data ($d > 20$), kNN suffers the curse of dimensionality: distances concentrate and discriminating power collapses. Reduce dimensionality with PCA or use a parametric / tree-based model.
- `kknn::train.kknn()` implements kernel-weighted kNN (closer neighbours weighted more heavily); often slightly more accurate than uniform-weight kNN.
- kNN is a lazy learner: prediction cost is $O(nd)$ per query for naive search; use `FNN` with kd-tree or ball-tree indexing for $O(\log n)$ queries on large training sets.
- For mixed numeric-categorical inputs, use Gower distance via `cluster::daisy()` instead of Euclidean; the standard distance breaks down on categorical features.

R Packages Used

`class::knn()` for the canonical implementation; `FNN::knn()` for fast kd-tree-based search; `kknn::train.kknn()` for weighted kNN with cross-validated k and kernel selection; `caret` and `tidymodels` for cross-validated kNN within ML pipelines; `RANN` for approximate nearest-neighbour search at scale.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP

- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis